COMP 412
FALL 2012

Simple Type Checking as an Application of Semantic Elaboration

Comp 412

Copyright 2012, Keith D. Cooper & Linda Torczon, all rights reserved.

Students enrolled in Comp 412 at Rice University have explicit permission to make copies of these materials for their personal use.

Faculty from other educational institutions may use these materials for nonprofit educational purposes, provided this copyright notice is preserved.

Why Do We Need Type Systems?

A type system creates a 2nd vocabulary to describe both the form and behavior of valid programs — one that can describe *context-sensitive properties* of the program.

A compiler typically used type information in three ways:

- Safety — to detect & avoid runtime errors
- Expressiveness — to allow statements beyond CFGs
- Runtime efficiency — to tailor generated code to contex

A type is just an abstract category for data that specifies properties held in common by all its members.

Why Do We Need Type Systems?

Safety

- Types help a compiler detect & avoid runtime errors
  - Dereferencing an out-of-bounds pointer
  - Interpreting an integer as a floating-point number
  - Cannot detect all il-formed programs
    - → *Set of ill-formed programs is not computable*
- Compiler must infer a type for each expression
- Compiler must check type of each value at each use
  - Looking for uses that are forbidden or that require conversion
  - (If language requires (and most do)) types guide automatic insertion of conversions (called *implicit conversions*)
- A type system is *sound* if it assigns an unambiguous type to every expression without the need for dynamic checks

Why Do We Need Type Systems?

Safety

- A language where every expression can be assigned an unambiguous type is said to be *strongly typed*
  - Every expression can be typed at compile time ⇒ statically typed
  - Some types are not known until run-time ⇒ dynamically typed
  - Commercial: strongly-typed languages lead to faster software development & greater faith in the results
- Alternatives
  - Untyped languages lack a type system or have just one type
    - → *assembly code or BCPL*
  - Weakly typed languages have poor type systems
    - → *Classic versions of C & C-derived languages*
    - → *The C standards committee has been trying to improve C's type system since its inception.*

Distinction between language and implementation of checking

Why Do We Need Type Systems?

Expressiveness

A well-designed type system allows the language designer to include features & behaviors that cannot be specified with context-free rules.

- Operator overloading is a classic example
  - "+" means addition, in any suitable type, along with implicit rules to convert between types that support addition
    - → *In BCPL, with 1 type, + is integer, #+ is floating point*
  - Method resolution in OOLs depends on type information
- Types make possible complex behaviors
  - Amount of an autoincrement or autodecrement in C
  - Default constructor or specialized one in Java
    - → *Type signature of argument list determines the choice*

Type Checking

Components of a Type System

- Base Types
- Rules for constructing new types
- An effective procedure for checking type equivalence
- Rules for inferring types of every expression

Examples

- ML & Modula-3 are strongly typed and, typically statically checked
- Common Lisp is strongly typed by needs dynamic checking
- ANSI C is a typed language, but many implementations do a poor job of identifying errors.
  - Situation has improved in the last decade, but is still not great

We teach type systems and type checking in depth in the 411/511/611 series of courses.

Type Checking

Components of a Type System

- Base Types
- Rules for constructing new types
- An effective procedure for checking type equivalence
- Rules for inferring types of every expression

Base Types

- Usually tied to data types supported by the processor
  - *integer, floating point, double precision, character, boolean*
- Some languages support higher-level base types
  - Fortran has complex numbers; lisp has rational numbers
  - Lisp has *cons* as a base type

Type Checking

Components of a Type System

- Base Types
- Rules for constructing new types
- An effective procedure for checking type equivalence
- Rules for inferring types of every expression

Constructed Types

- Rules to construct more complex objects from base types
  - Lisp allows (encourages?) construction of lists from *cons* cells
  - Arrays, strings, records, graphs, tuples, …
  - Many languages include enumerated types
  - Functions and methods have compound, constructed types
- Ability for a user to construct new types is powerful

Type Checking

Components of a Type System

- Base Types
- Rules for constructing new types
- An effective procedure for checking type equivalence
- Rules for inferring types of every expression

Type Equivalence

- *Name equivalence:* If the programmer calls two objects by the same name, the compiler trusts the programmer
- *Structural equivalence:* The compiler builds a representation of each constructed type and applies a well-defined procedure to compare two types

Type Checking

Components of a Type System

- Base Types
- Rules for constructing new types
- An effective procedure for checking type equivalence
- Rules for inferring types of every expression

Inference Rules

- Define result of an operation by types of its operands
  - Addition: *integer x real* → *real*
- Often includes a notion of compatibility
  - In Pascal, the range 0…100 is compatible with integer
- Often specified in tabular form
  - *And implemented via ad-hoc, syntax-directed translation*

Type Checking

Components of a Type System

- Base Types
- Rules for constructing new types
- An effective procedure for checking type equivalence
- Rules for inferring types of every expression

Inference Rules

- What about more complex type systems, such as ML's system
  - Build an explicit abstract syntax tree (using ad-hoc SDT)
  - Build an explicit representation for the types
  - Analyze type system outside the parser
    - → *Might be a more complex treewalk*
    - → *Might be an iterative fixed-point procedure*

We teach type systems and type checking in depth in the 411/511/611 series of courses.